

EventHub — Rapport de projet

Examen pratique DevOps · Master 1 Intelligence Artificielle **Dakar Institute of Technology** · Août 2026

Équipe 6

NOM	RÔLE	GITHUB
DIOUF François Pape	Scrum Master	@dioufra
Mohamed Sow	Développeur	@sultan2096
Berly Lora	Développeur	@loraham057-spec
Kra Junior	Développeur	@KraJunior
Aichatou Ndaw	Développeur	@chattoundaw26-tech

Dépôt : <https://github.com/dioufra/eventhub>

1. Introduction

1.1 Contexte

Le Dakar Institute of Technology organise régulièrement des conférences, des ateliers et des séminaires. Leur gestion reposait jusqu'ici sur des outils dispersés — Google Forms pour les inscriptions, Excel pour les listes, emails pour les échanges. Il en résultait trois difficultés : impossibilité de suivre les inscriptions en temps réel, absence de vision d'ensemble pour les organisateurs, et communication laborieuse avec les participants.

1.2 Objectif

EventHub réunit ces usages dans une plateforme unique permettant de créer et gérer des événements, d'administrer les comptes participants, de gérer les inscriptions, et de suivre en temps réel les places restantes et les statistiques.

1.3 Objectif pédagogique

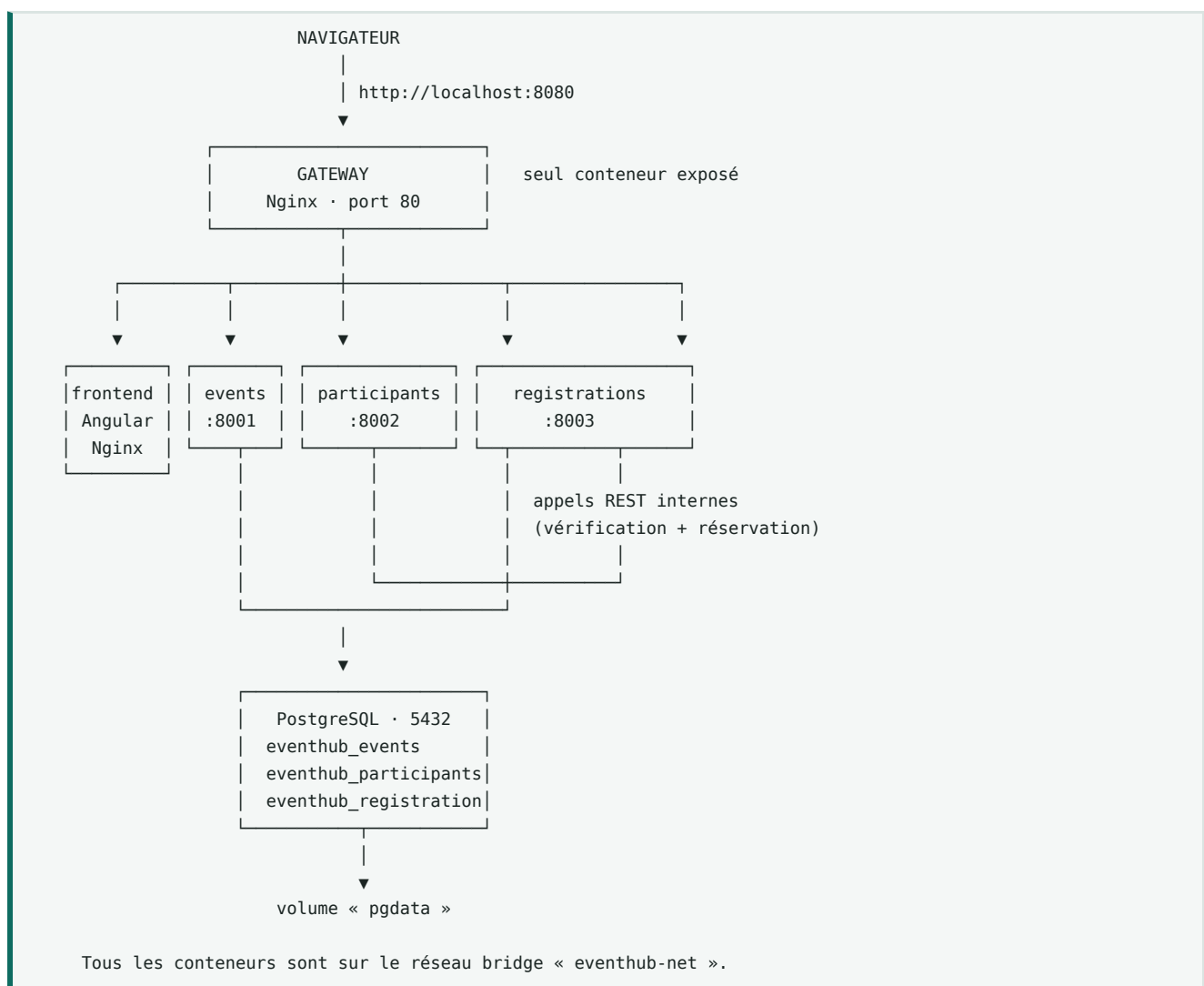
Au-delà du produit, le projet visait à mettre en pratique une chaîne DevOps complète : conception en microservices, conteneurisation, intégration et déploiement continu, et gestion de projet avec Git et GitHub.

1.4 Chiffres du projet

Microservices backend	3
Conteneurs applicatifs	6
Endpoints REST	25
Tests unitaires	48
Contrôles de recette	27
Pipelines GitHub Actions	2

2. Architecture

2.1 Vue d'ensemble



2.2 Décisions d'architecture

Une gateway dédiée, seul point d'entrée. Un conteneur Nginx reçoit tout le trafic et route selon l'URL : `/api/events` vers le service événements, le reste vers le frontend. Aucun autre conteneur ne publie de port.

Trois bénéfices. La **sécurité** d'abord : ce qui n'est pas exposé ne peut pas être attaqué depuis l'extérieur. L'absence totale de **CORS** ensuite, puisque le navigateur ne voit qu'une seule origine — c'est le problème numéro un des projets microservices, et il disparaît par construction. La **centralisation** enfin : en-têtes de sécurité, journalisation et limitation de débit se règlent à un seul endroit.

Une instance PostgreSQL, trois bases logiques. Le principe microservices impose qu'un service ne lise jamais les tables d'un autre. Trois bases distinctes — `eventhub_events`, `eventhub_participants`, `eventhub_registrations` — garantissent cette isolation. Nous avons mutualisé l'instance plutôt que d'en déployer trois : le bénéfice d'isolation est identique, pour un tiers des ressources. Passer à trois instances ne demanderait que trois blocs supplémentaires dans le fichier d'orchestration.

Conséquence directe : les colonnes `event_id` et `participant_id` de la table des inscriptions **ne sont pas des clés étrangères SQL** — elles ne peuvent pas l'être, les tables cibles vivant dans d'autres bases. Ce sont des références logiques, validées par appel HTTP au moment de l'inscription.

Pas de service discovery. Docker fournit nativement un serveur DNS interne : dans le réseau `eventhub-net`, le nom `events-service` résout vers l'adresse du conteneur. Un registre de services comme Consul n'apporterait rien tant qu'il n'y a pas plusieurs instances par service.

2.3 Le point délicat : la disponibilité des places

L'énoncé demande de vérifier les places restantes dans le service Événements, alors que les inscriptions vivent dans un autre service. Si chacun interroge l'autre, on crée une dépendance circulaire.

Notre solution maintient un flux **unidirectionnel**. Le service Événements détient un compteur `seats_taken` et expose deux endpoints internes que seul le service Inscriptions appelle :



Le service Inscriptions dépend des deux autres ; l'inverse n'est jamais vrai. Aucun cycle.

La réservation est protégée par un verrou SQL (`SELECT ... FOR UPDATE`). Sans lui, deux inscriptions simultanées sur la dernière place liraient toutes deux la même valeur et l'écriraient toutes deux : on obtiendrait 51 inscrits pour 50 places.

3. Description des microservices

Modèle de données détaillé : le document [MODELE-DONNEES.md](#) présente les deux vues complémentaires — le modèle conceptuel (domaine métier) et le modèle physique (répartition réelle en trois bases). La distinction est importante : le MCD laisserait croire à des clés étrangères entre les trois tables, alors qu'il n'en existe aucune.

Les trois services partagent la même organisation interne, ce qui permet de les relire sans effort et d'écrire un pipeline unique qui les traite tous par matrice.

```

<service>/
├─ app/
│   ├── main.py      point d'entrée, sondes de santé
│   ├── config.py    configuration par variables d'environnement
│   ├── database.py   connexion PostgreSQL
│   ├── models.py     tables SQLAlchemy
│   ├── schemas.py    validation Pydantic
│   └── routes.py     endpoints REST
└─ tests/
  
```

3.1 Service Événements — port 8001

Modèle de données — table `events`

COLONNE	TYPE	CONTRAINTE
id	integer	clé primaire
title	varchar(200)	non nul, 3 caractères minimum
description	text	facultatif
starts_at	timestamp	non nul, indexé
location	varchar(200)	non nul, indexé
capacity	integer	non nul, supérieur à 0
seats_taken	integer	non nul, défaut 0
created_at , updated_at	timestamp	automatiques

Les index sur `starts_at` et `location` correspondent aux filtres demandés.

Endpoints

MÉTHODE	CHEMIN	RÔLE
GET	/api/events	Lister, filtres <code>?date=</code> et <code>?location=</code>
POST	/api/events	Créer
GET	/api/events/{id}	Détails
PUT	/api/events/{id}	Modifier
DELETE	/api/events/{id}	Supprimer
GET	/api/events/{id}/availability	Places restantes
POST	/api/events/{id}/seats/reserve	(interne) Réserver une place
POST	/api/events/{id}/seats/release	(interne) Libérer une place

Règles métier — la capacité ne peut pas être ramenée sous le nombre d'inscrits (409) ; la réservation sur un événement complet est refusée (409) ; la libération ne descend jamais sous zéro.

3.2 Service Participants — port 8002

Modèle de données — table `participants`

COLONNE	TYPE	CONTRAINTE
id	integer	clé primaire
full_name	varchar(150)	non nul, indexé
email	varchar(150)	non nul, unique , indexé
phone	varchar(30)	facultatif
type	varchar(20)	etudiant professeur externe
created_at , updated_at	timestamp	automatiques

Endpoints

MÉTHODE	CHEMIN	RÔLE
GET	/api/participants	Lister, recherche ?search= sur nom ou email
POST	/api/participants	Créer
GET	/api/participants/{id}	Détails
PUT	/api/participants/{id}	Modifier
DELETE	/api/participants/{id}	Supprimer

L'unicité de l'email est garantie à deux niveaux : contrôle applicatif renvoyant un 409 explicite, et contrainte `UNIQUE` en base qui tient même si deux requêtes arrivent exactement en même temps.

3.3 Service Inscriptions — port 8003

Modèle de données — table `registrations`

COLONNE	TYPE	CONTRAINTE
id	integer	clé primaire
event_id	integer	non nul, indexé, référence logique
participant_id	integer	non nul, indexé, référence logique
status	varchar(20)	confirmed cancelled
created_at , updated_at	timestamp	automatiques

Endpoints

MÉTHODE	CHEMIN	RÔLE
GET	/api/registrations	Lister
POST	/api/registrations	Inscrire
DELETE	/api/registrations/{id}	Annuler
GET	/api/registrations/event/{id}	Inscriptions d'un événement
GET	/api/registrations/participant/{id}	Événements d'un participant
GET	/api/registrations/stats	Statistiques

C'est le seul service qui communique avec les autres. Son module `clients.py` encapsule ces appels, avec un **délai d'attente maximal de 5 secondes** : sans lui, un service bloqué bloquerait toute la chaîne jusqu'au navigateur.

L'annulation est **logique** : la ligne passe en `cancelled` plutôt que d'être supprimée, ce qui préserve l'historique pour les statistiques. Une seconde annulation est sans effet — la place n'est pas libérée deux fois.

3.4 Sondes de santé

Chaque service expose deux sondes, distinction reprise des standards de production :

SONDE	QUESTION	USAGE
/health	Le processus répond-il ?	HEALTHCHECK Docker
/health/ready	Le service et sa base répondent-ils ?	Supervision

/health/ready renvoie un 503 si PostgreSQL est injoignable. Nous n'utilisons volontairement pas cette sonde dans le `HEALTHCHECK` : une base momentanément lente ferait passer tous les conteneurs en `unhealthy` et déclencherait des redémarrages en cascade.

4. Dockerisation

Cinq images sont construites, une par composant applicatif, plus l'image officielle de PostgreSQL.

IMAGE	BASE	TAILLE FINALE
eventhub-events-service	python:3.12-alpine	193 Mo
eventhub-participants-service	python:3.12-alpine	193 Mo
eventhub-registrations-service	python:3.12-alpine	193 Mo
eventhub-frontend	nginx:1.30-alpine	94 Mo
eventhub-gateway	nginx:1.30-alpine	93 Mo

4.1 Choix des images de base

L'énoncé demande des images légères. Nous avons retenu **Alpine** partout.

Ce choix a une conséquence sur le pilote PostgreSQL. Le pilote le plus répandu, `psycopg2`, est écrit en C : sur Alpine il faudrait installer un compilateur et les en-têtes de la bibliothèque cliente PostgreSQL, soit environ 200 Mo supplémentaires. Nous avons retenu `pg8000`, un pilote entièrement écrit en Python, qui s'installe sans rien compiler. C'est ce qui permet de rester sur Alpine sans surcoût.

4.2 Build multi-étapes pour le frontend

Le frontend est le cas où le multi-étapes est le plus spectaculaire :

```
# ÉTAPE 1 – compilation avec Node
FROM node:20-alpine AS build
COPY package.json package-lock.json ./
RUN npm ci --legacy-peer-deps          # couche mise en cache
COPY . .
RUN npm run build -- --configuration=production

# ÉTAPE 2 – l'image finale ne garde que le résultat
FROM nginx:1.30.4-alpine
COPY --from=build /app/dist/frontend/browser /usr/share/nginx/html
```

L'étape de compilation a besoin de Node, du CLI Angular et d'environ 500 Mo de dépendances. L'image finale ne contient que des fichiers HTML, CSS et JavaScript : **94 Mo au lieu de plus de 600.**

4.3 Les choix appliqués à chaque Dockerfile

Ordre des instructions. Les dépendances sont copiées et installées avant le code applicatif. Docker met chaque couche en cache : comme `requirements.txt` change rarement et le code souvent, cette seule règle fait passer les reconstructions de plus d'une minute à quelques secondes.

Utilisateur non privilégié. Chaque image crée un utilisateur `appuser` et bascule dessus avant de lancer l'application. Si une faille était exploitée, l'attaquant ne serait pas `root` dans le conteneur.

Sonde de santé. Chaque image déclare un `HEALTHCHECK`. Ce n'est pas décoratif : l'orchestration utilise `depends_on: condition: service_healthy`, et un conteneur sans sonde ne devient jamais sain — la gateway ne démarrerait jamais.

Écoute sur toutes les interfaces. Les services sont lancés avec `--host 0.0.0.0`. Par défaut, uvicorn n'écoute que sur la boucle locale : le conteneur démarrerait normalement et refuserait toutes les connexions.

Sortie non tamponnée. `PYTHONUNBUFFERED=1` fait apparaître les journaux immédiatement dans `docker logs`, sans quoi ils arrivent avec du retard, voire pas du tout si le conteneur s'arrête brutalement.

.dockerignore. Chaque service exclut environnements virtuels, caches, tests et surtout le fichier `.env`. Sans cela, des identifiants pourraient se retrouver dans une image publiée.

4.4 Orchestration

Trois fichiers se combinent :

FICHIER	RÔLE
<code>docker-compose.yml</code>	Base commune : 6 services, réseau, volume
<code>docker-compose.override.yml</code>	Développement — chargé automatiquement
<code>docker-compose.prod.yml</code>	Production — à préciser explicitement

En développement, la surcharge publie les ports des services pour accéder à leur documentation interactive, et monte le code depuis l'hôte : une modification est prise en compte en une seconde au lieu de reconstruire l'image.

En production, la section `build` est neutralisée — on ne compile rien, on tire les images publiées — aucun port n'est publié sauf celui de la gateway, et des limites de processeur et de mémoire sont appliquées.

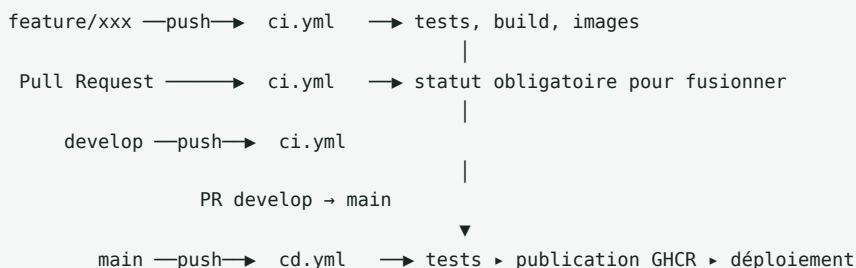
Ordonnancement du démarrage. Les dépendances utilisent `condition: service_healthy`. Sans cela, les services tenteraient de joindre PostgreSQL avant qu'il n'accepte les connexions. C'est la version déclarative de ce qu'on écrirait sinon à la main dans un script d'attente.

Persistance. Un volume nommé `pgdata` conserve les données. Nous l'avons vérifié en créant un événement, en supprimant les conteneurs par `docker compose down`, puis en les relançant : l'événement était toujours là.

Journalisation. Une politique de rotation limite chaque conteneur à 30 Mo de journaux. Sans elle, les fichiers peuvent saturer le disque d'un serveur en quelques jours.

5. Intégration et déploiement continus

Deux pipelines GitHub Actions, séparés parce qu'ils répondent à deux questions différentes : *le code est-il correct ?* et *comment le livrer ?*



5.1 Le pipeline d'intégration — `ci.yml`

JOB	RÔLE
compose	Valide l'orchestration, vérifie qu'un seul port est publié en production et que le script d'initialisation crée bien trois bases
backend	Tests unitaires des trois services, en matrice parallèle
frontend	Installation, compilation de production, vérification du chemin de sortie
images	Construction des cinq images, sans publication

Actions utilisées : `actions/checkout@v4`, `actions/setup-python@v5`, `actions/setup-node@v4`, `docker/setup-buildx-action@v3`, `docker/build-push-action@v6`.

Deux choix méritent d'être signalés.

La matrice. Le job de tests est écrit une fois et s'exécute trois fois en parallèle, une par service. C'est le bénéfice direct de la structure interne identique imposée aux trois services : sans elle, il aurait fallu écrire trois jobs.

Une CI progressive. Chaque job commence par vérifier si son périmètre est implémenté. Tant qu'un service est vide, l'étape est ignorée avec un message plutôt que de faire échouer le pipeline. Nous avons adopté cette conception au démarrage du projet, quand la plupart des fichiers étaient encore vides : une CI systématiquement rouge n'est plus lue par personne, et l'alerte se noie dans le bruit le jour où un vrai problème survient. Le pipeline s'est ainsi activé tout seul, service après service, sans qu'une seule ligne du fichier ait été modifiée.

5.2 Le pipeline de livraison — `cd.yml`

Déclenché sur `main` uniquement.

ÉTAPE	CONTENU
1. Tests	Rejeu des tests unitaires — on ne publie jamais sans revalider
2. Publication	Cinq images poussées sur GitHub Container Registry
3. Déploiement	Pile démarrée depuis les images publiées
4. Recette	27 contrôles de bout en bout

Pourquoi GHCR plutôt que Docker Hub. Le jeton `GITHUB_TOKEN` est fourni automatiquement par GitHub Actions : aucun compte supplémentaire, aucun secret à créer et donc aucun risque de fuite d'identifiants. Le job déclare `permissions: packages: write`, application du principe du moindre privilège.

Étiquetage des images. Chaque image reçoit deux étiquettes : `latest`, et `sha-<commit>`. La seconde est la plus précieuse — elle permet de répondre avec certitude à la question « quel code tourne exactement en production ? ».

Sur le déploiement. Ne disposant pas de serveur dédié, l'étape de déploiement s'exécute **sur le runner GitHub, à partir des images publiées sur GHCR**. La pile de production complète est réellement démarrée, attendue jusqu'à ce que les six conteneurs soient sains, puis soumise aux 27 contrôles de recette. Ce n'est donc pas une simulation : les images livrées sont effectivement déployées et éprouvées. Le script de déploiement par SSH vers un serveur réel figure en commentaire à la fin du fichier, prêt à être activé dès qu'une machine sera disponible.

Nous préférons documenter cette limite plutôt que de la masquer.

5.3 Gestion du projet

Stratégie de branches conforme à l'énoncé : `feature/*` pour le développement, `develop` pour l'intégration, `main` pour la production. Aucun commit n'a été poussé directement sur `main`.

Suivi par issues. Le backlog a été découpé en 52 issues, chacune reliée à une exigence de l'énoncé et suivie dans un tableau GitHub Projects avec des champs d'estimation, de priorité et de sprint. Chaque branche est créée depuis son issue par `gh issue develop`, ce qui lie automatiquement les deux, et chaque Pull Request ferme les issues correspondantes à la fusion.

5.4 Qualité et sécurité

MESURE	OÙ
Conteneurs non privilégiés	Chaque Dockerfile
Images de base minimales	Alpine partout
Build multi-étapes	Frontend
<code>.dockerignore</code> et <code>.gitignore</code>	Empêchent secrets et caches d'être diffusés
Aucun secret versionné	<code>.env</code> ignoré, <code>.env.example</code> fourni comme modèle
Un seul port exposé	<code>docker-compose.prod.yml</code>
<code>no-new-privileges</code>	Production
Limites processeur et mémoire	Production
Validation des entrées	Schémas Pydantic
Requêtes paramétrées	SQLAlchemy, protège des injections SQL
Délais d'attente sur les appels	<code>clients.py</code>
En-têtes de sécurité HTTP	Gateway
Rotation des journaux	Orchestration

6. Interface

L'interface est développée en **Angular 20** et servie sous forme de fichiers statiques par Nginx.

6.1 Génération automatique des clients d'API

Plutôt que d'écrire à la main les modèles TypeScript et les appels HTTP, nous les **générons depuis les spécifications OpenAPI** que FastAPI expose automatiquement.

```
npm run api:refresh    # récupère les 3 spécifications, régénère les clients
```

83 fichiers sont produits — modèles et services typés — et versionnés avec le projet.

Le bénéfice n'est pas le temps gagné, mais la **fiabilité** : si un champ change côté serveur, la régénération casse la compilation TypeScript au lieu de produire un bug silencieux à l'exécution. Aucune dérive n'est possible entre le contrat d'API et son utilisation.

Nous avons ajouté côté FastAPI une fonction de nommage des opérations, sans laquelle les méthodes générées portaient des noms illisibles : `listEventsApiEventsGet()` est ainsi devenu `listEvents()`.

6.2 Adresses relatives : le point de configuration décisif

Les clients générés utilisent par défaut l'adresse absolue `http://localhost` . En conteneur, le navigateur aurait appelé le port 80 de la machine de l'utilisateur au lieu de passer par la gateway.

Nous forçons une adresse de base **vide**, ce qui rend tous les appels relatifs (`/api/events`). En développement, le proxy du serveur Angular les redirige vers les ports 8001 à 8003 ; en conteneur, la gateway les route vers les microservices.

Conséquence importante : **la même image fonctionne en développement comme en production**, sans aucune variable d'environnement. C'est nécessaire parce qu'Angular compile ses fichiers au moment du build — une variable fournie au démarrage du conteneur n'aurait aucun effet.

6.3 Écrans

ÉCRAN	FONCTIONNALITÉS
Événements	Liste, filtres par date et par lieu, places restantes, état ouvert ou complet
Création d'événement	Formulaire avec validation
Participants	Liste, recherche par nom ou email, création
Inscriptions	Parcours guidé : événement, participant, résumé, confirmation

L'interface reprend l'identité visuelle du DIT et gère les états de chargement, d'erreur et de liste vide.

6.4 Captures d'écran

À insérer avant le rendu. Placer les fichiers dans `docs/captures/` .

CAPTURE	CONTENU ATTENDU
01-evenements.png	Liste des événements avec les places restantes
02-creation-evenement.png	Formulaire de création
03-participants.png	Liste et recherche
04-inscription.png	Parcours d'inscription avec le résumé
05-evenement-complet.png	Un événement complet, confirmation désactivée
06-compose-ps.png	<code>docker compose ps</code> — les 6 conteneurs sains
07-swagger.png	Documentation interactive d'un service
08-ci.png	Un run CI entièrement vert
09-cd.png	Un run CD jusqu'au déploiement
10-packages.png	Les 5 images publiées sur GHCR
11-smoke.png	Sortie de <code>make smoke</code> — 27 contrôles réussis
12-branches.png	Le graphe des branches (Insights → Network)

7. Difficultés rencontrées et solutions

7.1 Une redirection invisible qui cassait toutes les créations

Symptôme. Les lectures fonctionnaient à travers la gateway, mais les créations échouaient sans message clair.

Cause. La configuration Nginx déclarait `location /api/events/`, avec une barre oblique finale. Nginx applique alors une règle documentée mais peu connue : une requête vers `/api/events`, sans barre finale, reçoit une redirection permanente `301`. Or un `POST` redirigé en `301` est converti en `GET` par la plupart des clients, **et perd son corps de requête**. La création d'événement échouait donc silencieusement.

Résolution. Nous avons monté un banc d'essai avec des services factices pour observer le comportement réel :

AVANT	<code>POST /api/events</code> → HTTP 301	la requête n'atteint jamais le service
APRÈS	<code>POST /api/events</code> → HTTP 200	

Le correctif tient en un caractère : retirer la barre oblique finale des directives `location`.

Ce que nous en retenons. Un test de lecture ne prouve rien sur les écritures. C'est ce qui nous a conduits à écrire une recette qui exerce réellement chaque verbe HTTP.

7.2 Deux conventions d'URL incompatibles

Symptôme. Le service Inscriptions répondait parfaitement à ses tests unitaires, mais renvoyait 404 pour tous les appels passant par la gateway.

Cause. Le service exposait ses routes sous `/registrations/...` tandis que la gateway transmettait `/api/registrations/...`. Les tests unitaires attaquant le service en direct, ils ne pouvaient pas détecter l'écart.

Résolution. Alignement sur une convention unique : le préfixe `/api` est porté par le service lui-même, et la gateway ne réécrit aucune URL. L'adresse appelée par le navigateur est exactement celle que reçoit le service, ce qui simplifie considérablement le diagnostic.

Nous avons ajouté un test dédié dans chaque service, qui vérifie que les routes répondent bien sous `/api/` et pas ailleurs.

7.3 Des tests verts qui auraient dû être rouges

Symptôme. Les tests d'un service passaient en local et échouaient en intégration continue.

Cause. La création des tables était appelée au niveau du module, donc exécutée dès l'import. Les tests tentaient ainsi de joindre PostgreSQL avant même de commencer. En local la base tournait, ce qui masquait le problème ; sur le runner, aucune base n'est disponible.

Résolution. Déplacement de cette création dans l'événement de démarrage de l'application. Les tests n'ouvrent plus aucune connexion réelle et utilisent une base SQLite en mémoire.

Vérification. Nous avons arrêté PostgreSQL et relancé les suites : les 48 tests passent sans aucune base, ce qui correspond aux conditions du runner.

7.4 Un conteneur qui ne devenait jamais sain

Symptôme. La gateway refusait de démarrer.

Cause. L'orchestration attend `condition: service_healthy` sur ses dépendances. L'un des Dockerfile ne déclarait pas de `HEALTHCHECK` : son conteneur ne pouvait donc jamais être considéré comme sain, et la gateway attendait indéfiniment.

Résolution. Ajout d'une sonde à chaque image.

7.5 Un test qui ne testait rien

Symptôme. Un contrôle de recette attendait un 409 et recevait un 422.

Cause. Pour vérifier qu'on ne peut pas réduire la capacité sous le nombre d'inscrits, nous envoyions une capacité de zéro — valeur déjà rejetée par la validation, avant d'atteindre la règle métier. Le code était correct ; le test, lui, ne prouvait rien.

Résolution. Reformulation du scénario : un événement à deux places, deux inscrits, puis une tentative de réduction à une place. La règle est désormais réellement éprouvée, et le cas passant a été ajouté.

7.6 Concilier les emplois du temps de l'équipe

La difficulté la plus structurante n'était pas technique. Entre les cours, les stages et les projets d'autres modules, les créneaux de travail des cinq membres se recoupaient rarement. Une organisation classique — tout le monde avance en parallèle, on synchronise en fin de journée — n'était pas applicable.

Ce que nous avons mis en place.

Un **découpage par périmètre exclusif** : chaque membre possède un dossier — un microservice, le frontend, l'infrastructure — et personne d'autre n'y touche. Cela supprime les conflits Git, mais surtout cela permet d'avancer sans attendre la disponibilité d'un autre.

Des **contrats d'API figés dès le premier jour**, avant toute ligne de code. Le service Inscriptions a ainsi pu être développé et testé contre des appels neutralisés, alors que les deux services qu'il interroge n'existaient pas encore. Sans ce contrat écrit, il aurait fallu attendre — et cette attente aurait été fatale au planning.

Une **structure interne identique** imposée aux trois microservices. Le premier service a demandé une journée complète ; le deuxième, construit sur le même moule, quelques heures. C'est aussi ce qui a permis d'écrire un pipeline d'intégration unique traitant les trois services par matrice, plutôt que trois jobs distincts.

Une **intégration continue progressive**, qui ignore proprement les périmètres non encore livrés au lieu d'échouer. Chaque membre pouvait ainsi pousser son travail sans casser le pipeline des autres, et les tests s'activaient automatiquement à mesure que le code arrivait.

La **discipline des branches et des Pull Requests** a été maintenue en toutes circonstances, y compris pour les tâches menées sans binôme : l'énoncé l'évalue explicitement, et la revue croisée reste le meilleur moyen d'attraper une erreur avant qu'elle n'atteigne la branche principale.

Ce que nous en retenons. Une architecture pensée pour l'indépendance des composants ne sert pas qu'à la production : elle rend une équipe aux disponibilités décalées capable de livrer. Le découpage en microservices, les contrats d'API et l'uniformité des structures ont eu autant de valeur organisationnelle que technique.

8. Améliorations possibles

8.1 Migrations de schéma

Les tables sont créées au démarrage si elles n'existent pas. Cette approche ne gère pas l'évolution du schéma : ajouter une colonne à une base contenant déjà des données demanderait une intervention manuelle. **Alembic** versionnerait les migrations et permettrait les retours arrière.

8.2 Transactions distribuées

Notre inscription réserve une place puis enregistre en base. Si l'écriture échoue, une transaction compensatoire libère la place. Ce mécanisme couvre le cas courant, mais pas tous : si le service tombe entre les deux opérations, la place reste réservée sans inscription correspondante.

Le motif **Saga**, avec un journal d'événements persistant, garantirait la cohérence même en cas d'arrêt brutal.

8.3 Communication asynchrone

Les appels entre services sont synchrones : si le service Événements est indisponible, aucune inscription n'est possible. Un bus de messages (RabbitMQ, Kafka) permettrait de découpler ces échanges — le service Inscriptions accepterait la demande et la traiterait dès que possible.

8.4 Authentification

L'application n'a aujourd'hui aucun contrôle d'accès : n'importe qui peut créer ou supprimer un événement. Un fournisseur d'identité OAuth2 / OpenID Connect, tel que **Keycloak**, sécuriserait les endpoints d'administration. La gateway est déjà le point de passage obligé : c'est là que la vérification des jetons se brancherait, sans modifier les microservices.

8.5 Orchestration à plus grande échelle

Docker Compose convient parfaitement à un serveur unique, mais ne gère ni la haute disponibilité, ni la montée en charge automatique, ni les déploiements progressifs.

Kubernetes apporterait réplicas, redémarrages automatiques et déploiements sans interruption de service.

8.6 Observabilité

Nos journaux et nos sondes sont locaux à chaque conteneur. Une stack **Prometheus, Grafana et Loki**, complétée par un traçage distribué **OpenTelemetry**, donnerait une vue de bout en bout : on pourrait suivre une inscription à travers les trois services et identifier précisément où le temps est passé.

8.7 Couverture de tests

Nous couvrons l'unitaire, plus une recette de bout en bout. Deux niveaux manquent : des **tests d'intégration** avec une vraie base PostgreSQL (Testcontainers), et des **tests de bout en bout dans un navigateur** (Playwright ou Cypress) pour valider les parcours utilisateur.

8.8 Déploiement progressif

Le déploiement actuel remplace directement les conteneurs, avec une brève interruption. Une stratégie **blue-green** ou **canary** permettrait de basculer sans coupure et de revenir en arrière instantanément.

8.9 Sécurité renforcée

Trois pistes : un **utilisateur PostgreSQL par service**, avec accès à sa seule base — application stricte du moindre privilège ; une **limitation de débit** au niveau de la gateway ; et une **analyse de vulnérabilités** des images, avec Trivy, intégrée au pipeline.

9. Conclusion

EventHub répond aux exigences de l'énoncé : trois microservices communiquant en REST, une base relationnelle, un `Dockerfile` par composant, une orchestration complète et un pipeline d'intégration et de déploiement continus.

Sur le plan technique, deux décisions se sont révélées structurantes. La **gateway comme point d'entrée unique** a supprimé par construction toute la problématique CORS et centralisé la sécurité. La **structure interne identique imposée aux trois services** a permis d'écrire un pipeline unique par matrice et de dupliquer un service en une fraction du temps nécessaire à sa création — ce qui s'est avéré décisif compte tenu des emplois du temps décalés de l'équipe.

Le projet nous laisse surtout une leçon de méthode : les bugs les plus coûteux n'ont pas été détectés par les tests unitaires. La redirection `301` de la gateway, le désaccord de préfixe d'URL, les tests non hermétiques — tous passaient les contrôles automatisés et n'ont été révélés que par une exécution réelle en conteneurs. C'est ce qui nous a conduits à écrire une recette de 27 contrôles qui exerce l'application déployée, et à l'intégrer au pipeline de livraison.
